Least-Squares Temporal Difference Learning

Reimplementing Boyan (ICML 1999)
and a ridge-regularised extension

Dinesh Chand, Meena Chidambram, Nadiya R S and Sabin

IISc, Mtech AI 2026

Problem statement and motivation

- An agent moves from state to state and receives a reward after each move. A policy is the rule that chooses those moves.
- In this project the policy is already chosen and does not change, so the system is a Markov reward process (MRP).
- If every transition probability were known, that estimate would be the solution of a linear system.
- The probabilities are unknown. We only see sampled trajectories, so the system has to be built from data.
- TD(λ) builds it through repeated step-size updates. The step size is chosen by hand, and a poor choice makes learning slow or unstable.
- Boyan (ICML 1999, **Best Paper Award**) replaces those updates with one least-squares solve, LSTD(λ). The same trajectories give a more accurate value, and there is no step size.
- With only a few trajectories the least-squares matrix can be poorly determined. **Ridge regression**, $(A + \epsilon I)\boldsymbol{\theta} = b$, is our addition for that case.

Policy evaluation

Exact value method

- The value of a state is its expected total future reward:

$$V^{\pi}(s) = \mathbb{E}\Big[\sum_{t\geq 0} \gamma^t r_t \;\Big|\; s_0 = s\Big]$$

- The Bellman equation splits that into one reward plus the rest:

$$V(s) = \mathbb{E}\big[r + \gamma V(s')\big]$$

- When P and the rewards are known, this is a linear system and can be solved directly.

Value-function approximation

- In this project the transitions are unknown. Each step is only a sample $(s,\, r,\, s_{\text{next}},\, \text{done})$.
- One stored number per state does not scale, so d weights represent every state, with $d \ll |S|$:

$$\hat{V}(s) = \boldsymbol{\phi}(s)^{\top}\boldsymbol{\theta}, \qquad \boldsymbol{\phi}(s) \in \mathbb{R}^d$$

- The features $\boldsymbol{\phi}(s)$ are fixed in advance. Monte Carlo, TD, and LSTD are three ways to learn $\boldsymbol{\theta}$.

Monte Carlo, TD, and LSTD

All three approximate the same value function. Write $\hat{V}(s) = \boldsymbol{\phi}(s)^\top \boldsymbol{\theta}$ and $\boldsymbol{\phi}_t = \boldsymbol{\phi}(s_t)$.

Monte Carlo

The target is the return actually received. The update waits until the episode ends.

$$G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$$

$$e_t = G_t - \hat{V}(s_t)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, e_t\, \boldsymbol{\phi}_t$$

Uses the observed outcome. One odd episode can move $\hat{V}$ a long way.
$\lambda = 1$ is this end of the scale.

TD(λ)

The target is one reward plus a discounted estimate of the next state.

$$\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$$

$$\mathbf{z}_t = \gamma\lambda\, \mathbf{z}_{t-1} + \boldsymbol{\phi}_t$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \mathbf{z}_t$$

$\lambda = 0$: only the current state.
Larger λ: earlier states keep a decaying share.
α sets how far each nudge goes.

LSTD(λ)

Same δ_t and $\mathbf{z}_t$. Solve for the $\boldsymbol{\theta}$ at which the updates add to zero.

$$\Delta_t = \boldsymbol{\phi}_t - \gamma \boldsymbol{\phi}_{t+1}$$

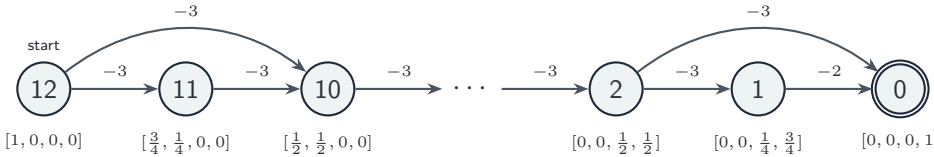
$$A \mathrel{+}= \mathbf{z}_t \Delta_t^\top$$

$$b \mathrel{+}= r_t\, \mathbf{z}_t$$

$$\boldsymbol{\theta} = A^{-1} b$$

No step size.
Terminal step: $\boldsymbol{\phi}_{t+1} = \mathbf{0}$.
LSTD(1) = Monte Carlo regression.

Boyan chain and the paper's experiment



The chain (paper Fig. 4)

- States $12 \to 0$. Start at 12, state 0 ends the episode, $\gamma = 1$.
- From $s \geq 2$: move to $s-1$ or $s-2$, each with probability $\frac{1}{2}$, reward -3.
- From 1: always move to 0, reward -2.
- True answer: $V^\pi(s) = -2s$, so $V(12) = -24$ and $V(0) = 0$.

What the paper measures (Sec. 4)

- Learn $\boldsymbol{\theta}$ from sampled episodes. Compare $\boldsymbol{\phi}(s)^\top \boldsymbol{\theta}$ with $-2s$.
- Algorithms: LSTD(λ) and offline TD(λ), on the same episodes.
- Sweep λ and the number of trajectories, up to 10^4.
- TD also sweeps step sizes. LSTD has none.
- Four features, anchored at 12, 8, 4, and 0. V^π is **exactly representable**: $\boldsymbol{\theta}^* = (-24, -16, -8, 0)$.

Parameters and variables

These are the objects in the equations, and the settings of the experiment we reimplement. Later slides use the same names.

Symbol	Code	Role
γ	`gamma`	Discount; 1 in this project
λ	`lam`	Trace decay, in $[0, 1]$
$\boldsymbol{\phi}(s)$	`phi[s]`	Feature row of state s
$\boldsymbol{\theta}$	`theta`	Weights in $\hat{V} = \boldsymbol{\phi}^\top \boldsymbol{\theta}$
$\mathbf{z}_t$	`z`	Eligibility trace
δ_t	`delta`	TD error
A, b	`A, b`	System LSTD solves
α	`alpha`	TD step size
ϵ	`reg`	Ridge, $(A + \epsilon \mathrm{I})\boldsymbol{\theta} = b$

Setting	Value in the replication
Features	$d = 4$; states between anchors are mixtures
λ grid	$0, 0.2, \ldots, 1$; headline plots use 0.4
Trajectories	1 to 10^4 episodes on each curve
TD schedules	constant α, and $\alpha_n = a_0 \dfrac{n_0 + 1}{n_0 + n}$
One transition	$(s,\ r,\ s_{\text{next}},\ \text{done})$
Terminal step	$\boldsymbol{\phi}(s_{\text{next}}) = \mathbf{0}$
Error	RMSE of $\boldsymbol{\phi}^\top \boldsymbol{\theta}$ vs. V^π, all states

04

TD(λ)

- Member B

From TD(0) to TD(λ)

TD(0): nudge $\boldsymbol{\theta}$ toward a one-step target

$$\delta_t = r_t + \gamma\, \boldsymbol{\phi}(s_{t+1})^\top \boldsymbol{\theta} - \boldsymbol{\phi}(s_t)^\top \boldsymbol{\theta}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \boldsymbol{\phi}(s_t)$$

TD(λ): credit each TD error to *all recent* states via an eligibility trace

$$\mathbf{z}_t = \gamma\lambda\, \mathbf{z}_{t-1} + \boldsymbol{\phi}(s_t), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \mathbf{z}_t$$

- $\lambda = 0$: one-step TD (low variance, biased)
- $\lambda = 1$: Monte Carlo (unbiased, high variance)
- In between: trades bias for variance

- $\mathbf{z}_t$ propagates reward backwards *without storing the trajectory*
- Cost: $O(K)$ per step

TD(λ) in the paper (Fig. 1) and step sizes

TD(λ), offline (paper Fig. 1)

for trajectory $n = 1, 2, \ldots$
  $\Delta \leftarrow 0$, $\mathbf{z} \leftarrow \boldsymbol{\phi}(s_0)$
  for each step t:
    $\Delta \leftarrow \Delta + \mathbf{z}\left(r_t + (\boldsymbol{\phi}_{t+1} - \boldsymbol{\phi}_t)^\top \boldsymbol{\theta}\right)$
    $\mathbf{z} \leftarrow \lambda \mathbf{z} + \boldsymbol{\phi}_{t+1}$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_n \Delta$

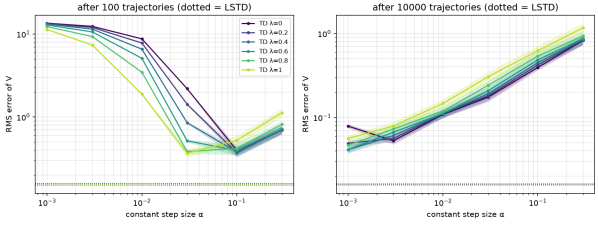
$\boldsymbol{\phi}(\text{end}) = 0$; $\boldsymbol{\theta}$ is fixed within a trajectory.

Step-size schedule (paper Eq. 8)

$$\alpha_n = a_0 \frac{n_0 + 1}{n_0 + n}$$

- $a_0 \in \{0.1,\ 0.01\}$
- $n_0 \in \{10^2,\ 10^3,\ 10^6\}$
- $\Rightarrow$ 6 schedules (paper)
- **We add** 6 constant $\alpha \in \{0.001, \ldots, 0.3\}$
- $\Rightarrow$ 12 TD configurations per λ

Weakness of TD: the step size



No single α is good both early and late ($\lambda = 0.4$, RMSE):

	$n = 100$	$n = 10^4$
TD $\alpha = 0.3$	0.70	0.85
TD $\alpha = 0.001$	13.2	0.041
TD, best sched.	0.31	0.036
LSTD	**0.15**	**0.016**

Other weaknesses (paper): sensitive to initial $\boldsymbol{\theta}$ and to feature scaling; each sample used once, then forgotten.

05

LSTD(λ)

- Member C

Where does TD(λ) converge?

- Averaged over trajectories, one TD(λ) pass changes $\boldsymbol{\theta}$ by

$$\mathbb{E}[\Delta\boldsymbol{\theta}] = \alpha\,(\mathbf{d} + C\boldsymbol{\theta}), \quad \mathbf{d} = \mathbb{E}\Big[\textstyle\sum_t \mathbf{z}_t r_t\Big], \quad C = \mathbb{E}\Big[\textstyle\sum_t \mathbf{z}_t(\gamma\boldsymbol{\phi}_{t+1} - \boldsymbol{\phi}_t)^\top\Big]$$

- TD stops moving when

$$\boxed{\mathbf{d} + C\boldsymbol{\theta} = 0}$$

- C is negative definite $\Rightarrow$ unique fixed point (Tsitsiklis & Van Roy).
- **TD is stochastic gradient descent on a $K \times K$ linear system** — it never writes the system down.

Idea: estimate $\mathbf{d}$ and C from data and solve the system directly.

The LSTD(λ) algorithm (paper Fig. 2)

LSTD(λ)

$A \leftarrow 0$, $\mathbf{b} \leftarrow 0$
for each trajectory:
  $\mathbf{z} \leftarrow 0$
  for each step t:
    $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \boldsymbol{\phi}_t$
    $A \leftarrow A + \mathbf{z}\,(\boldsymbol{\phi}_t - \gamma\boldsymbol{\phi}_{t+1})^\top$
    $\mathbf{b} \leftarrow \mathbf{b} + \mathbf{z}\,r_t$
whenever needed: $\boldsymbol{\theta} = A^{-1}\mathbf{b}$

- After n trajectories: $\mathbf{b} \approx n\mathbf{d}$, $A \approx -nC$
- $\Rightarrow A^{-1}\mathbf{b}$ estimates TD's fixed point
- **No step size, no initial θ**
- Invariant to feature scaling
- Keeps *all* data in A, $\mathbf{b}$
- Singular A (too little data): paper uses SVD; we use `solve` with an SVD fallback

Three views of LSTD(λ) (paper Secs. 2–3)

1. $\lambda = 0$**:** exactly Bradtke & Barto's LSTD (1996), with a much simpler derivation.
2. $\lambda = 1$**:** least-squares regression of features onto Monte-Carlo returns, built *incrementally* (telescoping: $A = \sum_t \boldsymbol{\phi}_t \boldsymbol{\phi}_t^\top$, $\mathbf{b} = \sum_t \boldsymbol{\phi}_t G_t$).
3. **Model-based view:** with one feature per state, LSTD(0) accumulates

$$A = N - C_{\text{count}}, \quad \mathbf{b} = \mathbf{s} \;\Rightarrow\; \boldsymbol{\theta} = (N - C_{\text{count}})^{-1}\mathbf{s},$$

which is exactly **"count transitions, build the empirical Markov chain, solve its Bellman equations"**.

Corridor example, one pass over the 8 episodes

LSTD(0): $\hat{V}(A) = 0.75$ (= TD's limit). LSTD(1): $\hat{V}(A) = 0$ (= MC). TD(0) needs $\sim$2000 replays with $\alpha = 0.01$ to reach 0.75.
All three equivalences are checked by unit tests in our repo.

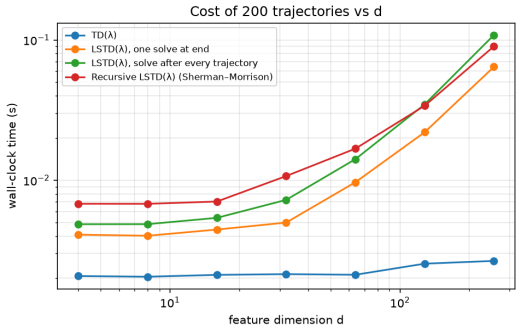
Computational cost and recursive LSTD

	per step	per solve
TD(λ)	$O(K)$	—
LSTD(λ)	$O(K^2)$	$O(K^3)$
Recursive LSTD	$O(K^2)$	—

Recursive LSTD keeps $B = A^{-1}$ up to date (Sherman–Morrison), $\mathbf{u} = \boldsymbol{\phi}_t - \gamma\boldsymbol{\phi}_{t+1}$:

$$B \leftarrow B - \frac{B\mathbf{z}\,\mathbf{u}^\top B}{1 + \mathbf{u}^\top B\mathbf{z}}$$

Start $B_0 = \delta I \Leftrightarrow$ ridge with $\epsilon = 1/\delta$.



At $d = 256$: TD 2.7 ms vs. LSTD 64 ms (200 trajectories).

Experimental setup

Reproduction (paper Sec. 4)

- Boyan chain, $\lambda \in \{0, 0.2, \ldots, 1\}$
- Up to 10^4 trajectories
- LSTD vs. 12 TD configurations
- RMSE over all 13 states
- **20 seeds** (paper: 10), mean $\pm$ 1 SE
- Same episodes for every algorithm within a seed (common random numbers)

Implementation

- Python + NumPy, CPU only
- One module per member, frozen interfaces
- 47 unit tests, incl. paper's equivalences
- Config-driven runner, full sweep $\approx$ 30 s on a laptop

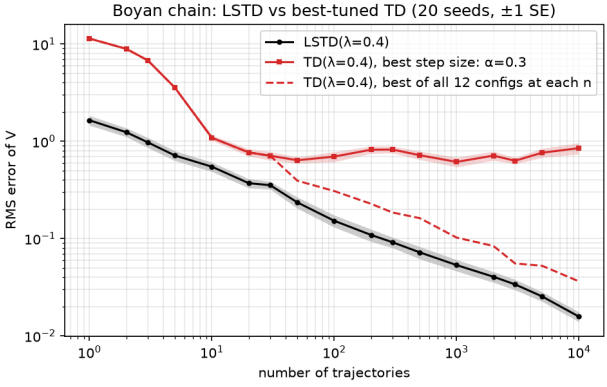
Fair comparison: TD is shown with its *best* step size — even an oracle that picks the best of 12 configurations separately at every n.

06

Experiments & novelty

- Member D

Headline result: LSTD is more data-efficient



$1.64 \rightarrow 0.016$

LSTD RMSE from 1 to 10^4 trajectories

≈ 0.7

where the best single TD α (0.3) plateaus

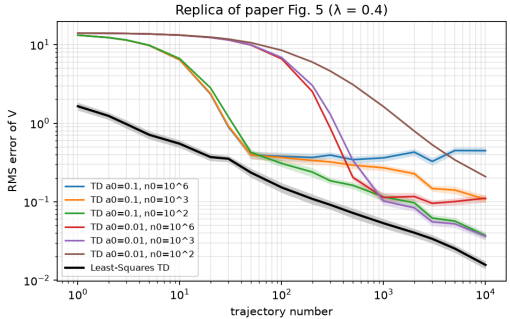
$\approx 2\times$

lower error than *oracle-tuned* TD (dashed) at every n

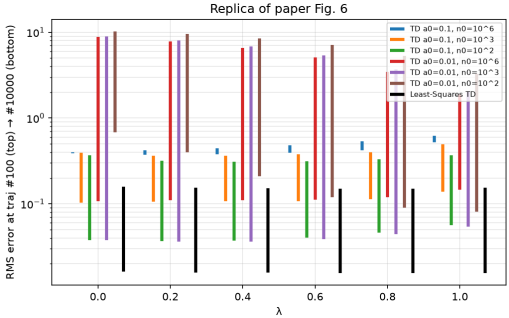
3–4×

fewer trajectories for the same error

Reproducing the paper's Figures 5 and 6



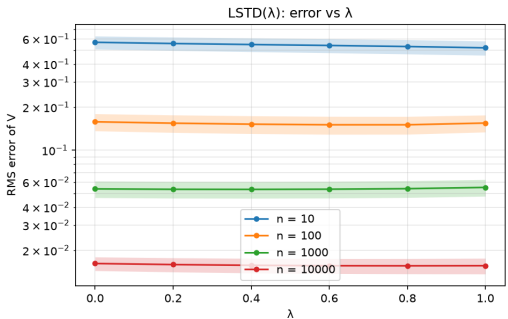
Paper Fig. 5: $\lambda = 0.4$, six schedules of Eq. 8



Paper Fig. 6: error at $n = 100$ (top) $\rightarrow 10^4$ (bottom)

Same conclusion as the paper: LSTD is best at every λ; TD depends critically on the schedule.

Effect of λ



	$n = 100$		$n = 10^4$	
λ	LSTD	TD*	LSTD	TD*
0.0	**0.158**	0.368	**0.016**	0.038
0.2	**0.155**	0.318	**0.016**	0.036
0.4	**0.152**	0.308	**0.016**	0.036
0.6	**0.151**	0.312	**0.016**	0.039
0.8	**0.151**	0.329	**0.016**	0.044
1.0	**0.155**	0.366	**0.016**	0.054

TD* = best of 12 TD configs for that n and λ.

- λ barely matters for LSTD here (features are exact)
- LSTD beats oracle-tuned TD by 2–$3.5\times$ everywhere

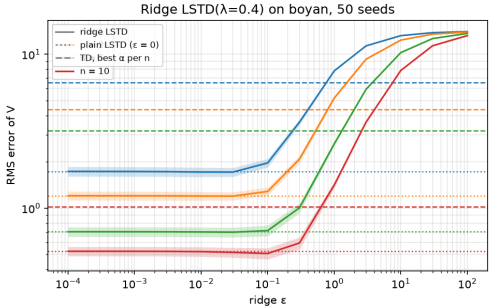
Novelty: ridge LSTD for very few trajectories

- With 1–10 trajectories, A can be singular or ill-conditioned $\Rightarrow \boldsymbol{\theta} = A^{-1}\mathbf{b}$ is unstable.
- **Ridge LSTD:**

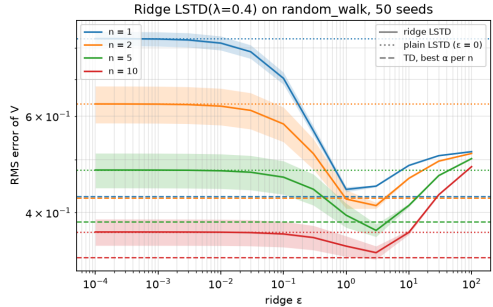
$$(A + \epsilon I)\,\boldsymbol{\theta} = \mathbf{b}$$

  (equivalently: recursive LSTD with $B_0 = I/\epsilon$).
- **Questions:**
  1. How does the error depend on ϵ for $n \in \{1, 2, 5, 10\}$?
  2. Can a small ϵ make LSTD competitive with the best-tuned TD at 1–5 trajectories?
  3. What happens beyond the paper: **19-state random walk** (Sutton & Barto) with **weak features** (5 aggregated groups, V^π not representable)?
- 50 seeds, $\lambda = 0.4$, $\epsilon \in [10^{-4}, 10^{2}]$; TD reference = best of 5 constant α at each n.

Novelty: results



Boyan chain: almost no gain (1–3% at $\epsilon \approx 0.03$). A is already well conditioned; shrinking toward 0 hurts when V is down to -24. Plain LSTD already beats TD (1.7 vs. 6.5 at $n = 1$).



Random walk, weak features: $\epsilon \approx 1$–3 **halves the error at** $n = 1$ (0.83 → 0.44), matching best-tuned TD (0.43); slightly better than TD at $n = 2, 5$; TD marginally better at $n = 10$.

Conclusions and limitations

Conclusions

- **Reproduced the paper:** LSTD beats TD at every n, λ and step size (2–$3.5\times$ below oracle-tuned TD).
- TD's best α depends on the data budget; a bad α costs 10–$100\times$.
- LSTD: no step size, insensitive to λ here.
- Price: $O(K^2)$ per step vs. $O(K)$.
- Ridge helps when data are scarce *and* features weak.

Limitations

- Toy chains; Boyan features are exact (best case for LSTD).
- TD tuned over a 12-point grid only.
- Fixed ϵ weakens as n grows (A is a sum).
- Ridge shrinks toward $\boldsymbol{\theta} = 0$, not a prior.

Next: Mountain Car with tile coding (large K) · LSPI: LSTD for control · clipped / sign TD-error baselines

Thank you

Questions?

</> src/ ▪ tests/ ▪ results/figures/

Backup: likely questions

Why more data-efficient? It keeps every transition in A, $\mathbf{b}$ and solves exactly. TD uses each sample for one small step and then forgets it. LSTD is effectively model-based.

What if A is singular? Too little data or redundant features. Paper: SVD pseudo-inverse. Us: `solve` with SVD fallback, or ridge $(A + \epsilon I)$.

Cost in K? $O(K^2)$ per step, $O(K^3)$ per solve (or $O(K^2)$ per step with recursive LSTD), vs. $O(K)$ for TD. Many features + cheap data $\Rightarrow$ TD can win on wall-clock time.

LSTD with $\lambda = 1$? Linear regression of features onto Monte-Carlo returns.

Control? Yes: LSTD-Q inside policy iteration = LSPI (Lagoudakis & Parr, 2003).

Is the comparison fair? Same episodes per seed; TD gets its best of 12 configs at every n.

Why does ridge not help on Boyan? A is well conditioned, and shrinkage biases $\boldsymbol{\theta}$ toward 0 while $\boldsymbol{\theta}^* = (-24, -16, -8, 0)$.

Backup: why LSTD(1) = Monte-Carlo regression

With $\lambda = \gamma = 1$, $\mathbf{z}_t = \sum_{i \le t} \boldsymbol{\phi}_i$ and $\boldsymbol{\phi}_T = 0$ at termination:

$$A = \sum_t \mathbf{z}_t (\boldsymbol{\phi}_t - \boldsymbol{\phi}_{t+1})^\top = \sum_i \boldsymbol{\phi}_i \sum_{t \ge i} (\boldsymbol{\phi}_t - \boldsymbol{\phi}_{t+1})^\top = \sum_i \boldsymbol{\phi}_i \boldsymbol{\phi}_i^\top$$

$$\mathbf{b} = \sum_t \mathbf{z}_t r_t = \sum_i \boldsymbol{\phi}_i \sum_{t \ge i} r_t = \sum_i \boldsymbol{\phi}_i G_i$$

So $\boldsymbol{\theta} = A^{-1}\mathbf{b}$ is ordinary least squares of G_i on $\boldsymbol{\phi}_i$ — built incrementally without storing trajectories.

Backup: code $\leftrightarrow$ paper

Paper	Code	Owner
Fig. 4 chain and features	`env.BoyanChain`, `features.boyan_features`	A
Exact V^π	`env.true_values` (Bellman solve)	A
Fig. 1 TD(λ), Eq. 8	`td_lambda.td_lambda_curve`, `step_size`	B
Fig. 2 LSTD(λ)	`lstd.lstd_lambda_curve`, `solve_theta`	C
Recursive LSTD (Sec. 2)	`lstd.recursive_lstd`	C
Sec. 3 model-based view	`tests/test_lstd.py`	C
Figs. 5, 6 and our figures	`run_experiment.py`, `plot_results.py`	D

```
python -m src.run_experiment && python -m src.plot_results
```